

Industry Learning Notes

Personal Study Notes — Days 0 to 3
Phase 1: LLM Foundations
Student: Sara | Mentor: AI Professor

Phase	Topics	Duration
0 — Python Gaps	Decorators, async, type hints, APIs	~1 week
1 — LLM Foundations	Transformers, Prompting, Embeddings, LangChain	3-4 weeks
2 — RAG Systems	Vector DBs, chunking, retrieval, evaluation	4-5 weeks
3 — Agentic AI	ReAct, tools, LangGraph, multi-agent	5-6 weeks
4 — Deployment	FastAPI, Docker, observability, cost	3-4 weeks
5 — Fine-tuning	LoRA, QLoRA, PEFT, when to fine-tune	2-3 weeks

Table of Contents

Introduction & Roadmap — Goals, Schedule, Learning Strategy

Day 0 — Setup — Environment, API keys, Project structure

Day 1 — LLM Foundations — Transformers, Tokens, Temperature, First API call

Day 2 — API & Chatbot — Response object, Memory, Sliding window

Day 3 — Prompt Engineering — 6 techniques, Structured output, JSON pipeline

Quick Reference — Python GenAI template, Error codes, Cheatsheet

Introduction & Learning Strategy

Your Starting Point

Python: Basics to intermediate — loops, OOP, lambda, file/exception handling, logging

DL/NLP Theory: Tokenization, stemming, BOW, TF-IDF, Word2Vec, ANN, CNN, RNN, LSTM, optimizers, loss functions, backprop

Practical Experience: None yet — this course bridges theory to industry practice

Python Gaps to Fill (Contextually)

- Decorators & closures — needed for tools and agents
- Async/await — needed for production APIs
- Type hints & Pydantic — needed for every LangChain object
- requests/httpx — needed for raw API calls

Core Learning Philosophy

- ✓ Correct Order: Foundations first, frameworks second. Never start with LangChain.
- ✓ No Copy-Paste Engineering: Understand every line before you write it.
- ✓ Concept First, Then Code: Every session follows this pattern.
- ✓ Build to Understand: Every concept ends with a working implementation.
- ✓ Token Awareness: Always know what you're sending and what it costs.

Why NOT to Start with LangChain

Most YouTube courses and commercial tutorials start with LangChain because it produces impressive demos quickly. This is optimised for views, not for your career. LangChain is an abstraction layer. If you learn abstractions before understanding what is being abstracted, you become a copy-paste engineer who cannot debug, optimise, or architect real systems. Learn the raw API first. Understand tokens, context windows, and response objects. Then LangChain will make complete sense when we reach it.

Day 0 — Environment Setup

Project Structure (Correct)

```
genai-learning/
├── .env # API keys – never commit this
├── .gitignore # Must be at project root
├── .venv/ # Created by UV – ignore in git
├── Day-1/
│   ├── test_connection.py
│   └── Day-2/
│       ├── chatbot.py
│       └── Day-3/
│           └── Assignment3.py
```

Setup Commands

```
# Create and activate environment
uv venv
.venv\Scripts\activate # Windows
source .venv/bin/activate # Mac/Linux

# Install packages
uv pip install groq python-dotenv
```

Key Lesson — .gitignore Placement

UV automatically creates a .gitignore INSIDE .venv/ to protect the environment folder. This does NOT protect your .env file. You must manually create a .gitignore at the PROJECT ROOT (same level as .env) containing: .env, .venv/, __pycache__/, *.pyc

API Provider Decision

Provider	Free Tier	Recommended Model
Groq	Yes — generous	llama-3.1-8b-instant
Google Gemini	Yes — regional limits	gemini-2.0-flash
OpenAI	No — pay-as-you-go	gpt-4o-mini
Anthropic	No free API tier	claude-3-haiku

Day 1 — How LLMs Actually Work

Why Transformers Replaced RNNs/LSTMs

You derived these two problems yourself:

- Problem 1 — Information bottleneck: RNN hidden state is overwritten at every step. By token 50, information from token 1 is essentially gone. Backpropagation causes vanishing gradients — the gradient shrinks exponentially travelling back through time steps.
- Problem 2 — Sequential computation: LSTM processes token N only after token N-1. Cannot be parallelised. Training on long sequences is devastatingly slow.

The Transformer Solution — Self-Attention

Every token directly attends to every other token simultaneously — no chain, no hidden state being overwritten, fully parallelisable. The attention score between two tokens reflects their contextual relevance. Unlike Word2Vec (static vectors), Transformer embeddings are contextual — 'bank' near 'river' has a different vector than 'bank' near 'money'.

```
RNN/LSTM → Sequential, hidden state chain, forgets early tokens
```

```
Transformer → Parallel, every token sees every token, nothing forgotten
```

```
Self-Attention → The mechanism that makes this possible
```

Tokenisation

Text enters the model as tokens, not words. A token is a chunk of text determined by the model's vocabulary. Tokens can be full words, subwords, or punctuation. Context windows are measured in tokens, not words. GPT-4o has a 128,000 token context window.

```
"I love machine learning" → ["I", " love", " machine", " learning"] = 4 tokens
```

```
"Unbelievably" → ["Un", "believ", "ably"] = 3 tokens
```

Temperature and Top-P

```
Temperature = 0 → always pick highest probability token (deterministic)
```

```
Temperature = 1 → sample proportionally (balanced, default)
```

```
Temperature = 1.5 → flatten distribution (creative, unpredictable)
```

```
Top-P = 0.9 → sample only from tokens that make up top 90% of probability mass
```

```
ignores the long tail of unlikely tokens
```

```
Rule of thumb: Use temperature=0 for classification, structured output, factual tasks. Use temperature>0 for creative writing, brainstorming, open-ended tasks.
```

First API Call — Groq SDK

```
from dotenv import load_dotenv
```

```
from groq import Groq
```

```
import os
```

```
load_dotenv() # Always load .env first
```

```
client = Groq(api_key=os.getenv('GROQ_API_KEY'))
```

```
response = client.chat.completions.create(  
model='llama-3.1-8b-instant',  
messages=[  
{'role': 'system', 'content': 'You are a helpful assistant.'},  
{'role': 'user', 'content': 'What is a token in LLMs?'}  
],  
temperature=0.7,  
max_tokens=500  
)  
  
print(response.choices[0].message.content)
```

System Messages — Key Insight

A system message is NOT architecturally special. It is just text placed at the beginning of the context window before the user message. The model was fine-tuned to respect whatever appears in the system position. Crucially — system messages consume tokens from your context window just like everything else.

```
Context window (what the model actually sees):  
[SYSTEM TOKENS] You are an expert Python teacher...  
[USER TOKENS] What is a token in LLMs?  
[ASSISTANT TOKENS] (model predicts from here)
```

Day 2 — API Response Anatomy & Chatbot Memory

Full Response Object

```
ChatCompletion(  
  id='chatcmpl-...',  
  choices=[  
    Choice(  
      finish_reason='stop', # 'stop' | 'length' | 'tool_calls'  
      index=0,  
      message=ChatCompletionMessage(  
        content='...', # the actual reply text  
        role='assistant',  
        tool_calls=None, # populated when model calls a tool  
      )  
    ],  
  ),  
  model='llama-3.1-8b-instant',  
  usage=CompletionUsage(  
    prompt_tokens=56, # tokens YOU sent  
    completion_tokens=94, # tokens model generated  
    total_tokens=150 # sum — what you pay for  
  )  
)
```

finish_reason Values

Value	Meaning
stop	Model finished naturally
length	Hit max_tokens limit — response cut off
tool_calls	Model decided to call a tool (Phase 3)

HTTP Error Codes to Know

Code	Meaning	Your Fault?
400	Bad request — wrong parameters	Yes
401	Authentication error — wrong API key	Yes
429	Rate limit / quota exceeded	Usually
500	Server error — provider's fault	No

Chatbot Memory — How It Works

The model has NO memory between API calls. Every call is stateless. To simulate memory, you maintain a list of dictionaries in Python and send the entire conversation history in every API call.

```
# Memory IS a list of dicts
conversation_history = [
{'role': 'system', 'content': 'You are a helpful assistant.'},
{'role': 'user', 'content': 'My name is Sara.'},
{'role': 'assistant', 'content': 'Hello Sara! How can I help?'},
{'role': 'user', 'content': 'What did I tell you my name was?'},
# model sees ALL of the above and answers correctly
]
```

Memory Management Strategies

Strategy	How	Tradeoff
Sliding window	Keep last N messages only	Cheap but forgets
Summarisation	Compress old messages into summary	Moderate cost, retains history
Retrieval (RAG)	Fetch only history relevant to query	Smart, scalable
Hybrid	Summarise + retrieve + keep recent full	Best quality, most complex

Sliding Window Implementation

```
# Always keep system message + last 6 messages
messages = [conversation_history[0]] + conversation_history[-6:]

# Why not just conversation_history[-6:]?
# Because [-6:] might cut off the system message if it's far back.
# [conversation_history[0]] wraps the dict in a list for + concatenation.
```

Day 3 — Prompt Engineering

Core Insight

Prompt Engineering is not magic. It is structured communication with a probabilistic text predictor. You are manipulating the probability distribution of the model's output by shaping its input context. The goal is not short or long prompts — the goal is precise and unambiguous prompts.

The 6 Core Techniques

1. Zero-Shot

Ask directly with no examples. Works for clear, unambiguous tasks.

```
Classify this text as positive, negative, or neutral:  
"The product arrived late but worked perfectly."
```

2. Few-Shot

Provide examples before the question. The model learns the pattern from context (in-context learning). Shifts the probability distribution toward the demonstrated pattern.

```
"I loved this product." → positive  
"Terrible experience." → negative  
"The package arrived." → neutral  
Now classify: "Arrived late but worked perfectly."
```

3. Chain of Thought (CoT)

Force step-by-step reasoning before the final answer. Reasoning tokens become context for the answer token — better intermediate context produces better final answers. Use for complex multi-step problems.

```
Bad: "What is 17 x 24?" (model guesses immediately)  
Good: "Solve step by step: What is 17 x 24?"  
(model works through steps, then answers with full context)
```

4. Role Assignment

Give the model a persona in the system message. Shapes the style, depth, and vocabulary of responses without changing factual accuracy.

```
System: "You are an expert Python teacher who explains  
everything using simple analogies."
```

5. Structured Output

Force the model to return parseable data formats (JSON). Essential for production pipelines. Always set temperature=0 for structured output.

```
System: "Return only valid raw JSON with no markdown,  
no backticks, no explanation."  
Fields: sentiment, reasoning, confidence, summary
```

6. Constraint Specification

Specify exact allowed values, formats, and limits. Prevents label drift and ambiguous outputs. Be explicit about every field's format.

```
sentiment: must be exactly one of: Positive, Negative, Neutral
reasoning: one single sentence string (not a list)
confidence: float between 0 and 1
```

Production Lesson — LLM Output Reliability

LLMs are unreliable on ambiguous ground truth. Two key protections:

1. Always request a confidence score. Use it to route low-confidence outputs to human review.
2. Always wrap `json.loads()` in `try/except`. Malformed JSON will crash your pipeline otherwise.

Day 3 Assignment — Sentiment Pipeline

```
from dotenv import load_dotenv
from groq import Groq
import os, json

load_dotenv()
client = Groq(api_key=os.getenv('GROQ_API_KEY'))

conversation_history = [
    {'role': 'system', 'content': '''You are a sentiment analysis engine.
Return only valid raw JSON. No markdown, no backticks, no explanation.
Exactly four fields:
- sentiment: must be exactly one of: Positive, Negative, Neutral
- reasoning: one single sentence explaining the classification
- confidence: float between 0 and 1
- summary: one line summary of the review'''}}
]

while True:
    user_input = input('Enter product review: ')
    if user_input.lower() == 'quit': break

    conversation_history.append({'role': 'user', 'content': user_input})

    response = client.chat.completions.create(
        model='llama-3.1-8b-instant',
        messages=conversation_history,
        temperature=0,
        max_tokens=200
    )

    reply = response.choices[0].message.content
    conversation_history.append({'role': 'assistant', 'content': reply})

    try:
        parsed = json.loads(reply)
        print('Sentiment: ', parsed['sentiment'])
```

```
print('Reasoning: ', parsed['reasoning'])
print('Confidence: ', parsed['confidence'])
print('Summary: ', parsed['summary'])
if parsed['confidence'] < 0.85:
    print('→ Flag for human review')
else:
    print('→ Passed to dashboard')
except json.JSONDecodeError:
    print('Malformed JSON. Raw output:', reply)
```

Quick Reference Cheatsheet

GenAI Python Template

[illegible]

What's Next — Day 4 Preview

Pre-Day 4 Mental Model:
Query → embed to vector
10,000 docs → each embedded to vector
Find docs whose vectors are closest to query vector
= semantic search = foundation of RAG

Progress Tracker

Day	Topic	Status
0	Environment setup, API keys, project structure	✓ Done
1	Transformers, tokens, temperature, first API call	✓ Done
2	Response object, chatbot memory, sliding window	✓ Done
3	Prompt engineering, structured output, JSON pipeline	✓ Done
4	Embeddings, cosine similarity, semantic search	Up next
5	Vector databases — FAISS, ChromaDB, Pinecone	Upcoming
6	RAG from scratch	Upcoming
7	LangChain — now you're ready for it	Upcoming
8+	Agentic AI, LangGraph, multi-agent systems	Upcoming